MODULE 10 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Caching, Relational Data and Choosing a Store

RDS, Aurora, ElastiCache – and the constraints that decide between them when two would work.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 10 of 34

admin@sthithapragna.com

In a selection question, the wrong answers usually **work**. They lose on a constraint the scenario stated in passing.

Amazon RDS for developers

- Managed relational engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server and Aurora.
- **Multi-AZ** is for availability. A standby you cannot read from, failover by DNS.
- **Read replicas** are for scale. Readable, asynchronous, and up to 15 on Aurora.
- **IAM database authentication** issues a 15-minute token instead of a stored password.
- The connection is TCP and stateful, which is where serverless compute starts to struggle.

> ON THE EXAM
>
> Multi-AZ against read replicas is the classic pairing: availability against scale. On Aurora you get both, from the same cluster.

The Lambda and RDS problem

- Each concurrent execution environment opens its **own connection**. Concurrency multiplies them.
- Opening a connection costs tens of milliseconds, on every cold start.
- Databases cap connections. Exceed it and you get 'too many clients', not throttling.
- Reaching a private database means attaching the function to a **VPC**, losing internet access.
- Three answers exist: RDS Proxy, the Aurora Data API, or not using a relational database.

> ON THE EXAM
>
> This is module 3's scenario 1 restated as a design problem. Connection pooling assumes long-lived processes, and Lambda has none.

Amazon RDS Proxy

- A managed pool sitting between your application and RDS or Aurora.
- Thousands of Lambda environments share a small number of real database connections.
- Credentials come from **Secrets Manager**, and IAM authentication is supported.
- Failover is faster, because the proxy holds the connection while the database moves.
- It runs in a VPC, is billed per vCPU-hour of the database, and adds a little latency.

> ON THE EXAM
>
> The answer when the constraint is 'keep the existing code and the existing SQL'. The application still speaks the native protocol.

Amazon Aurora for developers

- MySQL and PostgreSQL compatible, with storage that grows automatically to 128 TB.
- **Cluster endpoints:** a writer endpoint, a reader endpoint that load-balances, and custom endpoints.
- Up to **15 replicas**, with replica lag usually in the low milliseconds.
- **Aurora Serverless v2** scales capacity in fine increments, and can now scale to zero.
- **Aurora Serverless v1 reached end of life on 31 March 2025.** Remaining clusters were upgraded to v2.

> ON THE EXAM
>
> Pointing read traffic at the writer endpoint wastes 15 replicas. Using the reader endpoint is the single easiest Aurora scaling win.

The Aurora Data API

- An **HTTPS endpoint** for running SQL. No driver, no connection, no pool.
- **No VPC required**, so the Lambda function keeps its internet access and its fast cold start.
- Authorised by **IAM**, with credentials read from Secrets Manager.
- Higher per-query latency than a native connection, and result size is limited.
- It is not a drop-in replacement. Existing code using a SQL driver must be rewritten.

> ON THE EXAM
>
> The Data API removes the connection problem rather than managing it. That is why it wins on 'no VPC' and loses on 'no code changes'.

RDS Proxy or the Data API?

RDS PROXY

Keep the code, pool the connections

- Native protocol and existing drivers
- No application changes at all
- Requires the function to be in a VPC
- Billed per vCPU-hour, always on
- Works with RDS and Aurora

AURORA DATA API

Remove connections entirely

- SQL over an HTTPS endpoint
- Application code must be rewritten
- No VPC needed for the function
- No standing charge
- Aurora only, not plain RDS

THE TIEBREAKER

'Minimal code change' points at RDS Proxy. 'The function must stay out of a VPC' or 'no idle cost' points at the Data API.

Amazon ElastiCache engines

- **The examinable pairing:** Redis against Memcached.
- **Valkey**, added October 2024, is a Linux Foundation fork of Redis 7.2 and a drop-in replacement.
- AWS now recommends Valkey for new deployments. It is around 20% cheaper on nodes.
- It exists because Redis changed its licence away from open source in March 2024.
- **Answer Redis on the exam.** Everything true of Redis is true of Valkey.

> ON THE EXAM
>
> This one is painless: Valkey is API-compatible with Redis, so the exam's Redis answers remain correct. Only the name and the price changed.

Redis or Memcached?

REDIS AND VALKEY

A data structure server

- Lists, sets, sorted sets, hashes
- Persistence and snapshots
- Replication and Multi-AZ failover
- Pub/sub and transactions
- The default choice

MEMCACHED

A simple object cache

- Strings and objects only
- No persistence at all
- No replication or failover
- Multi-threaded, so scales up well
- Scales out by adding nodes

THE TIEBREAKER

Any requirement beyond a plain key-value cache selects Redis. Memcached wins only on multi-threaded simplicity and pure horizontal scale-out.

Caching strategies

- **Lazy loading:** read the cache, and on a miss read the database and populate it.
- Only requested data is cached, and a cache failure is survivable. Every miss costs latency.
- **Write-through:** update the cache whenever you update the database.
- Data is never stale, and reads always hit. But you cache data nobody asks for.
- **A TTL** on every key bounds staleness whichever strategy you choose. Use both together.

> ON THE EXAM
>
> Lazy loading serves stale data until eviction; write-through wastes writes on cold data. Real systems use write-through plus a TTL, which is what DAX does.

ElastiCache beyond caching

- **Session store** for a stateless web tier, so any instance can serve any user.
- **Leaderboards**, using sorted sets to rank millions of scores in constant time.
- **Rate limiting**, using atomic counters with expiry per caller.
- **Distributed locks** and job queues, using SETNX and list operations.
- **Pub/sub** for lightweight real-time fan-out between application components.

> ON THE EXAM
>
> Any scenario mentioning sorted sets, leaderboards, rankings or sliding-window rate limits is pointing at Redis, not at a cache in front of a database.

The other purpose-built stores

SERVICE	RECOGNISE IT WHEN THE SCENARIO SAYS
Amazon DocumentDB	MongoDB compatible, existing MongoDB application
Amazon Neptune	Graph, relationships, social network, fraud rings
Amazon Keyspaces	Apache Cassandra compatible, CQL
Amazon Timestream	Time series, IoT metrics at scale
Amazon MemoryDB	Redis compatible, but as a durable primary database
Amazon OpenSearch	Full-text search, log analytics, fuzzy matching

ON THE EXAM

You will not be asked to configure these. You will be asked to recognise the one word in a scenario that names one of them.

Choosing a store

THE SCENARIO SAYS	THE INTENDED ANSWER
Known access patterns, any scale, single-digit ms	DynamoDB
Ad hoc queries, joins, reporting, existing SQL skills	RDS or Aurora
Sub-millisecond reads of DynamoDB, no code change	DAX
Sub-millisecond reads of anything else	ElastiCache
Session state for a stateless tier	ElastiCache
Large objects, files, media, backups	Amazon S3
Full-text or fuzzy search	OpenSearch, fed by Streams

ON THE EXAM

Worth screenshotting. Most store-selection questions on this exam reduce to one row of this table.

Six scenarios, because selection needs practice. For each, name the **disqualifying clause** before you pick.

A new serverless application uses Lambda functions that need to run SQL against an Aurora PostgreSQL cluster. The functions also call a third-party payment API over the internet on every request.

The team is writing the data access layer from scratch. They want the simplest architecture, no standing infrastructure cost while the application is idle, and cold starts kept short.

How should the Lambda functions reach the database?

A Attach the functions to the VPC and connect directly to the cluster endpoint

B Use the Aurora Data API over HTTPS, authorised with IAM

C Attach the functions to the VPC and connect through RDS Proxy

D Attach the functions to the VPC, add a NAT gateway, and connect through RDS Proxy

◆ PAUSE AND DECIDE

How should the Lambda functions reach the database?

A Direct VPC connection

Breaks the payment API call, because a VPC-attached function has no internet route. It also exhausts connections at concurrency.

B Aurora Data API

No VPC, so internet access survives and cold starts stay short. No standing cost, and the code is being written from scratch anyway.

C RDS Proxy in a VPC

Solves connections, still breaks the payment API, and adds a per-hour charge the team explicitly does not want while idle.

D RDS Proxy plus NAT gateway

Works fully, and adds two always-on billed components. It is the correct answer to a question with different constraints.

THE SIGNAL

Three clauses: an internet call, no idle cost, and code written from scratch. Each one favours the Data API, and together they eliminate everything else.

An existing PHP application runs on EC2 behind an Application Load Balancer and stores user session state in local files on each instance. Auto scaling is being introduced, and users are being logged out whenever an instance is replaced or traffic moves.

The team wants the smallest possible change to the application, and sessions must survive an instance failure.

Where should session state be stored?

A Amazon S3, writing a session file per user

B Sticky sessions on the Application Load Balancer, keeping local files

C An Amazon RDS table, with a row per session

D Amazon ElastiCache, using the engine's session handler

◆ PAUSE AND DECIDE

Where should session state be stored?

A S3 per-session file

S3 is object storage, not a session store. Latency is far too high for a read on every page request, and there is no expiry without lifecycle rules.

B Sticky sessions

The tempting one. Sticky sessions keep a user on one instance, but the session still dies when that instance is replaced - which is the stated failure.

C RDS session table

It works, and it puts high-frequency ephemeral writes on a relational database, adding load and cost for data that is discarded hourly.

D ElastiCache session handler

Purpose-built: sub-millisecond reads, native TTL for expiry, survives instance loss, and the session handler is a configuration change rather than a rewrite.

THE SIGNAL

Sticky sessions solve routing, not durability. The requirement is that sessions survive instance failure, and stickiness does nothing for that.

A mobile game must show a global leaderboard of the top 100 players out of 40 million, updated the moment a score changes, plus each player's own rank. Scores change thousands of times per second.

The current implementation stores scores in DynamoDB and runs a Scan with sorting in the application, which takes minutes and is always out of date.

How should the leaderboard be implemented?

A Amazon ElastiCache for Redis, using a sorted set keyed by score

B A DynamoDB global secondary index with score as the partition key

C DynamoDB Accelerator in front of the existing table

D Export the table to Amazon S3 and query it with Amazon Athena

◆ PAUSE AND DECIDE

How should the leaderboard be implemented?

A Redis sorted set

Sorted sets maintain ranking as an intrinsic property. Top-N and a single player's rank are both constant-time, and updates are immediate.

B GSI on score

Score is a terrible partition key - low cardinality and hot. A GSI also cannot return a global ranking without reading every partition.

C DAX

Caches reads of a query that is itself wrong. A cached Scan is a faster wrong answer, and the data would be stale the moment a score changed.

D Athena over S3

An analytics tool answering a real-time question. Export latency alone breaks the requirement for immediate updates.

THE SIGNAL

Ranking is a data structure problem, not a query problem. DynamoDB has no concept of global order across partitions; Redis has ranking built in.

> A finance team needs a reporting database for settled transactions. Analysts write their own SQL, joining transactions to accounts, customers and currency tables, and the queries are different every week. Volume is about 200 GB and grows slowly.
>
> The engineering team's instinct is DynamoDB, because the rest of the platform uses it.

Which data store fits the requirement?

A DynamoDB with a GSI for each known report

B DynamoDB with PartiQL, so analysts can write SQL-like queries

C Amazon Aurora PostgreSQL

D DynamoDB with DAX to speed up the reporting queries

◆ PAUSE AND DECIDE

Which data store fits the requirement?

A **A GSI per report**

Requires knowing every query in advance. The scenario says the queries differ every week, so this fails on the first new question asked.

B **PartiQL**

The best-built distractor. PartiQL gives SQL-like syntax but no joins, and a query without a key condition still becomes a full Scan.

C **Aurora PostgreSQL**

Ad hoc queries, multi-table joins and existing SQL skills are the definition of a relational workload. 200 GB is unremarkable for Aurora.

D **DAX**

Caches repeated reads. Queries that are different every week have no repetition to cache.

THE SIGNAL

Two phrases decide it: 'analysts write their own SQL' and 'different every week'. Both mean the access patterns cannot be known in advance, which is DynamoDB's one requirement.

> A product catalogue service caches product details in ElastiCache using lazy loading with a 24-hour TTL. Merchandisers report that price changes take up to a day to appear on the site, and complaints are increasing.
>
> The catalogue is read constantly but written rarely - a few hundred product updates per day against millions of reads. The team wants price changes visible within seconds without losing the cache's benefit.

What should the team change?

A Reduce the TTL to 30 seconds

B Add write-through: update the cache in the same code path that updates the product

C Remove the cache and read from the database on every request

D Increase the ElastiCache node size so more products stay cached

◆ PAUSE AND DECIDE

What should the team change?

A 30-second TTL

Fixes staleness by destroying the cache hit rate. Millions of reads would now miss every 30 seconds, pushing the load back onto the database.

B Write-through on update

A few hundred writes a day is a trivial amount of extra work, and it makes every price change visible immediately while the hit rate stays high.

C Remove the cache

Solves staleness by removing the thing that made the site fast. The scenario explicitly asks to keep the cache's benefit.

D Bigger nodes

Capacity was never the problem. Stale entries stay stale in a larger cache.

THE SIGNAL

The write-to-read ratio decides it. Write-through costs work proportional to writes, and here writes are rare while reads are enormous.

> A service caches rendered HTML fragments, keyed by URL. Fragments are regenerated cheaply if missing, are never read after the page changes, and no fragment matters individually. The cache must scale horizontally to tens of nodes as traffic grows, and the team wants maximum cache memory per pound spent.
>
> There is no requirement for persistence, replication, failover, or any data structure beyond a key and a blob.

Which ElastiCache engine fits best?

A Redis with cluster mode enabled and one replica per shard

B Redis with cluster mode disabled and five read replicas

C MemoryDB for Redis

D Memcached with nodes added as traffic grows

◆ PAUSE AND DECIDE

Which ElastiCache engine fits best?

A Redis, cluster mode enabled with replicas

Every replica is memory you pay for and never read from. The scenario says no fragment matters, so replication buys nothing.

B Redis, cluster mode disabled

Caps total cache size at one primary's memory, which directly contradicts scaling horizontally to tens of nodes.

C MemoryDB

A durable primary database with Multi-AZ transaction logs. Paying for durability on disposable HTML fragments is the opposite of the requirement.

D Memcached

Multi-threaded, scales out purely by adding nodes, and every gigabyte of node memory is cache rather than replica. Exactly the narrow band where Memcached wins.

THE SIGNAL

This is the deliberate mirror of the usual advice. The scenario removes every reason to choose Redis, one clause at a time - which is what makes Memcached correct.

What to carry forward

- Lambda plus a relational database has three answers: **RDS Proxy**, the **Data API**, or a different store.
- **Minimal code change** selects RDS Proxy. **No VPC or no idle cost** selects the Data API.
- Anything beyond a plain key-value cache selects **Redis**. Leaderboards and sessions are Redis questions.
- Write-through when writes are rare and reads are many. Always add a TTL.
- **Unknown or ad hoc access patterns rule DynamoDB out.** That is its one hard requirement.

> ON THE EXAM
>
> That closes the data stores. Next: Amazon API Gateway - the front door for most of what we have built so far.